
Producing Senders from Coroutine-Native Code

Document Number:	D4056R0
Date:	2026-03-13
Audience:	LEWG
Reply-to:	Vinnie Falco vinnie.falco@gmail.com Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Revision History

R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Bridge

3. The Three-Channel Problem

4. The Abstraction Floor

5. Above and Below

6. The Translation Layer

7. Conclusion

8. Acknowledgments

References

Appendix A. Bridge Implementation

Abstract

An `IoAwaitable` ([P4003R0](#)^[2]) can be wrapped as a `std::execution` sender. The adapter works. It also exposes the three-channel problem: an I/O awaitable returning `(error_code, size_t)` cannot be routed through `set_value` / `set_error` / `set_stopped` without losing information or bypassing composition. A compile-time constraint - a `static_assert` rejecting awaitables that return compound I/O results - eliminates the problem entirely. The constraint creates an abstraction floor. Above the floor, every awaitable returns void or

a single value, and the three channels map unambiguously. Below the floor, compound I/O results stay in coroutine-land where structured bindings handle them naturally. The coroutine body is the translation layer. This paper is informational and proposes no action.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.
-

1. Disclosure

The authors developed `Capy`^[3] and `Corosio`^[5]. `Capy`^[3] is a coroutine primitives library. `D4055R0`^[6] showed the sender-to-awaitable direction. This paper shows the reverse. The bridge depends on `Capy`^[3] and `beman::execution`^[4], a community implementation of `std::execution` (`P2300R10`^[1]). Source code links refer to pinned commit `60bf781`^[3]. The complete implementation is in Appendix A.

2. The Bridge

`as_sender` wraps any `IoAwaitable` as a `std::execution` sender. The receiver's environment carries the I/O execution context:

```
cap::thread_pool pool;

auto sndr = cap::as_sender(cap::delay(500ms));

auto op = ex::connect(
    std::move(sndr),
    demo_receiver{
        {pool.get_executor(), std::stop_token{}}},
    &done});

ex::start(op);
```

The receiver's environment answers a `get_io_executor` query with the pool's executor. The adapter's `start` extracts it, builds an `io_env`, and feeds it to the awaitable's `await_suspend(h, io_env const*)`. The awaitable runs on the pool. When it completes, the bridge calls `set_value` on the receiver.

```
main thread: 4256
  starting delay...
  set_value on thread 43448
  delay completed
```

The delay ran on thread 43448, a pool worker. The bridge allocated zero bytes beyond the coroutine frame.

3. The Three-Channel Problem

The bridge works for `delay`. What about `read_some`?

`read_some` returns `io_result<size_t>` - an `(error_code, size_t)` pair. The adapter must route this through three channels. D4053R1^[7] documented the trade-off for sender-based I/O. The same trade-off appears here:

- Route the whole `io_result` through `set_value`. The sender's completion signature becomes `set_value_t(io_result<size_t>)`. The error code is buried inside the struct. `when_all` does not cancel siblings on I/O failure. `upon_error` is unreachable. The three-channel composition algebra is bypassed.
- Decompose it. Call `set_value(size_t)` on success, `set_error(error_code)` on failure. The byte count is discarded on error. A `write` that sends 500 of 1,000 bytes before `ECONNRESET` produces `set_error(ECONNRESET)`. The 500 is gone.

Neither option preserves both values and retains composition. This is the same finding as D4053R1^[7], now appearing inside the bridge adapter itself.

4. The Abstraction Floor

The solution is to not bridge at this level. The adapter rejects it at compile time:

```

template<class IoAw>
auto as_sender(IoAw&& aw)
{
    using R = decltype(
        std::declval<std::decay_t<IoAw>&>()
            .await_resume());
    static_assert(
        !detail::is_io_result_type<
            std::decay_t<R>>::value,
        "as_sender does not accept IoAwaitables "
        "that return io_result. Wrap the operation "
        "in a task<T> that throws on error, "
        "then bridge the task.");
    return awaitable_sender<std::decay_t<IoAw>>{
        std::forward<IoAw>(aw)};
}

```

The `static_assert` creates an abstraction floor. `IoAwaitables` whose `await_resume` returns `io_result` are rejected. The bridge only accepts void or single-value returns. The three-channel problem cannot arise because the types that cause it are excluded from the bridge.

5. Above and Below

The floor divides the awaitable space into two regions.

Above the floor - bridgeable:

Awaitable	await_resume type
<code>delay(500ms)</code>	<code>void</code>
<code>async_event::wait()</code>	<code>void</code>
<code>async_mutex::lock()</code>	<code>void</code>
<code>task<int></code>	<code>int</code>
<code>task<std::string></code>	<code>std::string</code>
<code>task<></code>	<code>void</code>

Every row returns void or a single value. The three channels map unambiguously: `set_value` for the result, `set_error` for exceptions, `set_stopped` for cancellation.

Below the floor - not bridgeable directly:

Awaitable	await_resume type
<code>stream.read_some(buf)</code>	<code>io_result<std::size_t></code>
<code>read(stream, buf)</code>	<code>io_result<std::size_t></code>
<code>write(stream, buf)</code>	<code>io_result<std::size_t></code>
<code>stream.connect(ep)</code>	<code>io_result<></code>

Every row returns a compound result. The three-channel problem lives here. The `static_assert` keeps it here.

6. The Translation Layer

To use I/O in a sender pipeline, wrap it in a `task<T>` that converts the error code to an exception:

```

copy::task<std::size_t>
read_all(auto& stream, auto buf)
{
    auto [ec, n] = co_await copy::read(
        stream, buf);
    if (ec)
        throw std::system_error(ec);
    co_return n;
}

auto sndr = copy::as_sender(
    read_all(stream, buf))
| ex::then([](std::size_t n) {
    std::cout << "read " << n
               << " bytes\n";
});

```

The `task<std::size_t>` returns a single value. It lives above the floor. The `co_await copy::read(stream, buf)` returns `io_result<std::size_t>`. It lives below the floor. The coroutine body is the translation layer between the two regions. The programmer writes the error-to-exception conversion

explicitly, at the call site, with full access to both the error code and the byte count.

The compiler enforces the boundary. A programmer who forgets the wrapping step and writes

```
as_sender(stream.read_some(buf))
```

 gets a compile error, not a silent loss of error information.

7. Conclusion

The three-channel problem is not a defect of the sender model. It is a consequence of bridging at the wrong abstraction level. Compound I/O results - `(error_code, size_t)` - do not fit three channels without loss. Single values and void fit perfectly.

The `static_assert` makes the cost visible. Low-level I/O primitives cannot cross into sender-land directly. Higher-level operations - wrapped in a `task<T>` that throws on error - can. The coroutine body is where the translation happens. The compiler enforces it.

When the bridge respects the abstraction floor, the three-channel problem vanishes.

8. Acknowledgments

The authors thank Dietmar Kühl for `beman::execution` ^[4].

References

1. [P2300R10](https://wg21.link/p2300r10) - “std::execution” (Michał Dominiak et al., 2024). <https://wg21.link/p2300r10>
2. [P4003R0](https://wg21.link/p4003r0) - “IoAwaitable” (Vinnie Falco, 2026). <https://wg21.link/p4003r0>
3. [cppalliance/capy](https://github.com/cppalliance/capy) - Coroutine primitives library. Commit 60bf781.
<https://github.com/cppalliance/capy/tree/60bf781d09024ce0f9cbd1505684249184331692>
4. [bemanproject/execution](https://github.com/bemanproject/execution) - Community implementation of `std::execution`.
<https://github.com/bemanproject/execution>
5. [cppalliance/corosio](https://github.com/cppalliance/corosio) - Coroutine-native networking library. <https://github.com/cppalliance/corosio>
6. [D4055R0](https://wg21.link/d4055r0) - “Consuming Senders from Coroutine-Native Code” (Vinnie Falco, Steve Gerbino, 2026).
<https://wg21.link/d4055r0>

7. **D4053R1** - "Sender I/O: A Constructed Comparison" (Vinnie Falco, Steve Gerbino, 2026).

<https://wg21.link/d4053r1>

Appendix A. Bridge Implementation

```
#include <boost/capy/concept/io_awaitable.hpp>
#include <boost/capy/ex/executor_ref.hpp>
#include <boost/capy/ex/io_env.hpp>
#include <boost/capy/io_result.hpp>

#include <beman/execution/execution.hpp>

#include <concepts>
#include <coroutine>
#include <exception>
#include <stop_token>
#include <type_traits>
#include <utility>

namespace boost::capy {

struct get_io_executor_t
{
    template<class Env>
    auto operator()(
        Env const& env) const noexcept
        -> decltype(env.query(
            std::declval<
                get_io_executor_t const&>()))
    {
        return env.query(*this);
    }
};

inline constexpr get_io_executor_t
    get_io_executor{};

struct io_sender_env
{
    executor_ref io_executor;
    std::stop_token stop_token;

    auto query(
        get_io_executor_t const&)
        const noexcept -> executor_ref
    {
        return io_executor;
    }
}
```

```

    auto query(
        beman::execution::get_stop_token_t
            const&) const noexcept
        -> std::stop_token
    {
        return stop_token;
    }
};

namespace detail {

template<class T>
struct is_io_result_type
    : std::false_type {};

template<class... Ts>
struct is_io_result_type<io_result<Ts...>>
    : std::true_type {};

template<class IoAw, class Receiver>
struct bridge_task
{
    struct promise_type;
    using handle_type =
        std::coroutine_handle<promise_type>;

    struct promise_type
    {
        io_env const* env_ = nullptr;

        bridge_task get_return_object() noexcept
        {
            return bridge_task{
                handle_type::from_promise(
                    *this)};
        }

        std::suspend_always
        initial_suspend() noexcept
        {
            return {};
        }

        std::suspend_always
        final_suspend() noexcept
        {

```

```

        return {};
    }

    void return_void() noexcept {}
    void unhandled_exception() noexcept {}

    template<class A>
    struct transform_awaiter
    {
        std::decay_t<A>& aw_;
        promise_type* p_;

        bool await_ready() noexcept
        {
            return aw_.await_ready();
        }

        decltype(auto) await_resume()
        {
            return aw_.await_resume();
        }

        auto await_suspend(
            std::coroutine_handle<> h)
            noexcept
        {
            return aw_.await_suspend(
                h, p_->env_);
        }
    };

    template<class A>
    auto await_transform(A&& a)
    {
        return transform_awaiter<A>{
            a, this};
    }
};

handle_type h_{};

~bridge_task()
{
    if(h_)
        h_.destroy();
}

```

```

    bridge_task() noexcept = default;

    bridge_task(bridge_task&& o) noexcept
        : h_(std::exchange(o.h_, {}))
    {
    }

    bridge_task& operator=(
        bridge_task&& o) noexcept
    {
        if(h_)
            h_.destroy();
        h_ = std::exchange(o.h_, {});
        return *this;
    }

    bridge_task(
        bridge_task const&) = delete;
    bridge_task& operator=(
        bridge_task const&) = delete;

private:
    explicit bridge_task(
        handle_type h) noexcept
        : h_(h)
    {
    }
};

} // namespace detail

template<class IoAw>
struct awaitable_sender
{
    using sender_concept =
        beman::execution::sender_t;

    using result_type = decltype(
        std::declval<std::decay_t<IoAw>&>()
            .await_resume());

    static auto make_sigs()
    {
        if constexpr (
            std::is_void_v<result_type>)
            return beman::execution::
                completion_signatures<

```

```

        beman::execution::
            set_value_t(),
        beman::execution::
            set_error_t(
                std::exception_ptr),
        beman::execution::
            set_stopped_t()>{};
    else
        return beman::execution::
            completion_signatures<
                beman::execution::
                    set_value_t(result_type),
                beman::execution::
                    set_error_t(
                        std::exception_ptr),
                beman::execution::
                    set_stopped_t()>{};
}

using completion_signatures =
    decltype(make_sigs());

IoAw aw_;

template<class Receiver>
struct op_state
{
    using operation_state_concept =
        beman::execution::operation_state_t;

    IoAw aw_;
    Receiver rcvr_;
    io_env env_;
    detail::bridge_task<IoAw, Receiver>
        bridge_;

    op_state(IoAw aw, Receiver rcvr)
        : aw_(std::move(aw))
        , rcvr_(std::move(rcvr))
    {
    }

    op_state(op_state const&) = delete;
    op_state(op_state&&) = delete;
    op_state& operator=(
        op_state const&) = delete;
    op_state& operator=(

```

```

op_state&&) = delete;

void start() noexcept
{
    auto renv =
        beman::execution::get_env(
            rcvr_);
    auto ex = get_io_executor(renv);

    std::stop_token st;
    if constexpr (requires {
        { renv.query(
            beman::execution::
                get_stop_token_t{}) }
        -> std::convertible_to<
            std::stop_token>; })
    {
        st = renv.query(
            beman::execution::
                get_stop_token_t{});
    }

    env_ = io_env{ex, st, nullptr};

    bridge_ =
        [](IoAw aw, Receiver rcvr)
        -> detail::bridge_task<
            IoAw, Receiver>
    {
        try
        {
            if constexpr (
                std::is_void_v<
                    result_type>)
            {
                co_await std::move(aw);
                beman::execution::
                    set_value(
                        std::move(rcvr));
            }
            else
            {
                auto result =
                    co_await
                        std::move(aw);
                beman::execution::
                    set_value(

```

```

        std::move(rcvr),
        std::move(
            result));
    }
}
catch(...)
{
    beman::execution::
        set_error(
            std::move(rcvr),
            std::current_exception
                ());
}
}(std::move(aw_),
    std::move(rcvr_));

    bridge_.h_.promise().env_ =
        &env_;
    bridge_.h_.resume();
}
};

template<class Receiver>
auto connect(Receiver rcvr) &&
    -> op_state<Receiver>
{
    return op_state<Receiver>(
        std::move(aw_),
        std::move(rcvr));
}

template<class Receiver>
auto connect(Receiver rcvr) const&
    -> op_state<Receiver>
{
    return op_state<Receiver>(
        aw_, std::move(rcvr));
}
};

template<class IoAw>
auto as_sender(IoAw&& aw)
{
    using R = decltype(
        std::declval<std::decay_t<IoAw>&>()
            .await_resume());
    static_assert(

```

```

!detail::is_io_result_type<
    std::decay_t<R>>::value,
    "as_sender does not accept "
    "IoAwaitables that return io_result. "
    "Wrap the operation in a task<T> that "
    "throws on error, then bridge "
    "the task.");
return awaitable_sender<
    std::decay_t<IoAw>>{
        std::forward<IoAw>(aw)};
}

} // namespace boost::copy

```